AI 600 Deep Learning
Assignment 3 Report

Name: Noor Ul Ain Anwar
Student ID: 25280062

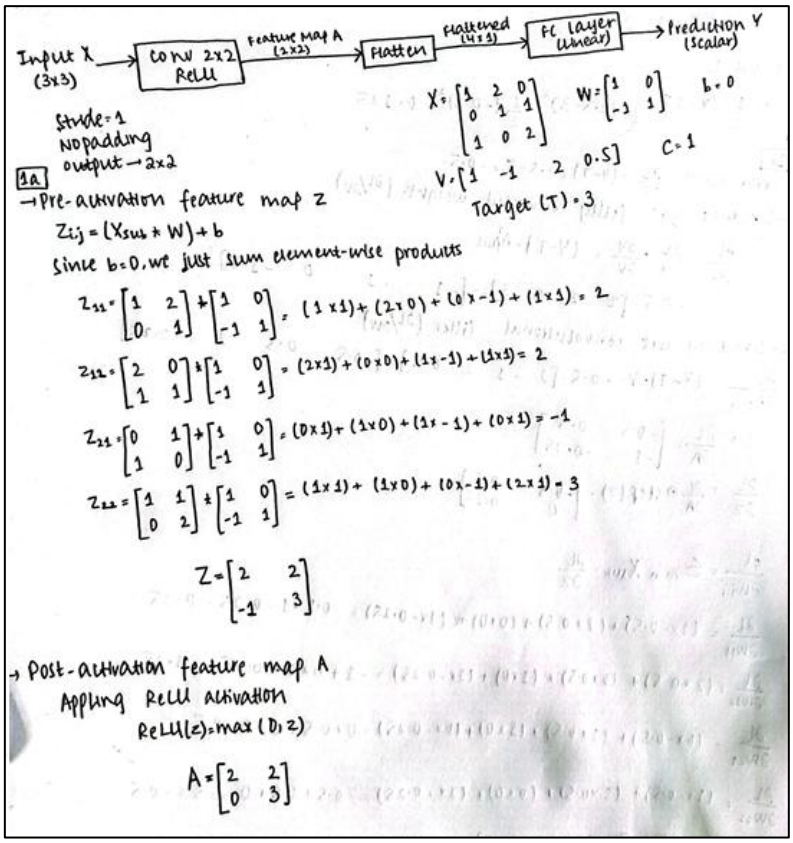
GitHub Repository Link: https://github.com/noorrulain/ai600-25280062-a3.git

---

Analytical

Question 1: Forward and Backward Pass

Part 1a



Stride = 1
No padding
output → 2x2

$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}$ $W = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$ $b = 0$

$V = [1 \quad -1 \quad 2 \quad 0.5]$ $c = 1$

Target (T) = 3

1A

→ Pre-activation feature map Z

$Z_{ij} = (X_{sub} * W) + b$

Since b = 0, we just sum element-wise products

$Z_{11} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = (1 \times 1) + (2 \times 0) + (0 \times -1) + (1 \times 1) = 2$

$Z_{12} = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = (2 \times 1) + (0 \times 0) + (1 \times -1) + (1 \times 1) = 2$

$Z_{21} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = (0 \times 1) + (1 \times 0) + (1 \times -1) + (0 \times 1) = -1$

$Z_{22} = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = (1 \times 1) + (1 \times 0) + (0 \times -1) + (2 \times 1) = 3$

$Z = \begin{bmatrix} 2 & 2 \\ -1 & 3 \end{bmatrix}$

→ Post-activation feature map A

Applying ReLU activation

$ReLU(z) = \max(0, z)$

$A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$

→ Final prediction Y

Flattening A

$A_{flat} = [2, 2, 0, 3]^T$

$V = [1 \quad -1 \quad 2 \quad 0.5]$ and $c = 1$

So,

$Y = (V \cdot A_{flat}) + c$

$= (1 \times 2) + (-1 \times 2) + (2 \times 0) + (0.5 \times 3) + 1$

$= 1.5 + 1 = 2.5$

→ Loss L

$L = \frac{1}{2}(Y - T)^2 = \frac{1}{2}(2.5 - 3)^2 = \frac{1}{2}(-0.5)^2 = 0.125$

Part 1b

$\boxed{1b}$

Error term $\frac{\partial L}{\partial Y} = (Y-T) = 2.5-3 = -0.5$

→ Gradient wrt fully connected weights $(\partial L/\partial V)$

$$\frac{\partial L}{\partial V} = \frac{\partial L}{\partial Y} \cdot \frac{\partial Y}{\partial V} = (Y-T) \cdot A_{flat}^{T}$$

$$= -0.5 \cdot [2 \quad 2 \quad 0 \quad 3] = [-1 \quad -1 \quad 0 \quad -1.5]$$

→ Gradient wrt convolutional filter $(\partial L/\partial W)$

$$\frac{\partial L}{\partial A_{flat}} = (Y-T) \cdot V = -0.5 \cdot [1 \quad -1 \quad 2 \quad 0.5] = [-0.5 \quad 0.5 \quad 1 \quad -0.25]$$

$$\hookrightarrow \frac{\partial L}{\partial A} = \begin{bmatrix} -0.5 & 0.5 \\ -1 & -0.25 \end{bmatrix}$$

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial A} \odot \text{step}(Z) = \begin{bmatrix} -0.5 & 0.5 \\ 0 & 0.25 \end{bmatrix}$$

$$\frac{\partial L}{\partial W_{i,j}} = \sum_{m,n} X_{sub} \cdot \frac{\partial L}{\partial Z}$$

$$\frac{\partial L}{\partial W_{11}} = (1 \times -0.5) + (2 \times 0.5) + (0 \times 0) + (1 \times -0.25) = -0.5 + 1 - 0.25 = 0.25$$

$$\frac{\partial L}{\partial W_{12}} = (2 \times -0.5) + (0 \times 0.5) + (1 \times 0) + (1 \times -0.25) = -1 + 0 + 0 - 0.25 = -1.25$$

$$\frac{\partial L}{\partial W_{21}} = (0 \times -0.5) + (1 \times 0.5) + (1 \times 0) + (0 \times -0.25) = 0 + 0.5 + 0 + 0 = 0.5$$

$$\frac{\partial L}{\partial W_{22}} = (1 \times -0.5) + (1 \times 0.5) + (0 \times 0) + (2 \times -0.25) = -0.5 + 0.5 + 0 - 0.5 = -0.5$$

$$\frac{\partial L}{\partial W} = \begin{bmatrix} 0.25 & -1.25 \\ 0.5 & -0.5 \end{bmatrix}$$

Part 1c

$\boxed{1c}$

The weight w_{12} has the largest absolute gradient $(|-1.25|)$ so it will change the most in gradient descent one step.
Since the gradient is negative and gradient descent moves in the direction of $-\eta \nabla L$, the weight will move in the positive direction. (increasing)
This specific weight has the largest impact because it was multiplied by the values in X that eventually passed through the ReLU and were scaled by the fully connected weights V. Considering V has a negative weight at the second position ($v_2 = -1$) and our prediction Y was lower than the target T, the network aims to increase Y. To increase Y when v_2 is negative, the network needs to decrease the activation a_{12}. Structurally, w_{12} was aligned with relatively high values in the input X at the positions that feed into the most sensitive parts of the loss calculation.

Question 3: Pooling and Normalization

Part 3a

3a

To recompute Y, instead of flattening and applying a FC layer, we apply a 2x2 global max pooling operation over A.

$$Y = \max(A) = \max[2 \quad 2 \quad 0 \quad 3]$$

$$Y = 3$$

Max pooling acts like a "router" during backward pass so the gradient only flows back through the specific node that was the maximum. In a1, the FC layer distributed the gradient to all elements in A because they were all multiplied by weights. Since $a_{22} = 3$ was the max then $\partial y / \partial a_{22} = 1$. All other activations have a gradient of exactly 0 because changing them slightly doesn't change the maximum value.

The gradient only flows back through a_{22} so only the pixels in the input image X that were used to compute a_{22} and z_{22} will receive a gradient. $\underline{z_{22}}$ is computed using 2x2 filter over bottom-right corner of X. pre-activation

$$X_{bottomright} = \begin{bmatrix} X_{2,2} & X_{2,3} \\ X_{3,2} & X_{3,3} \end{bmatrix}$$

Since only these 4 pixels have a gradient, all other pixels have a zero-gradient i.e, $X_{1,1} \mid X_{1,2} \mid X_{1,3} \mid X_{2,1} \mid X_{3,1}$.

Part 3b

3b

Mean of A (μ) $= \frac{1}{N} \sum a_i = \frac{2+2+0+3}{4} = 1.75$

Variance of A (σ^2) $= \frac{1}{N} \sum (a_i - \mu)^2 = \frac{1}{4}\left[(2-1.75)^2 + (2-1.75)^2 + (0-1.75)^2 + (3-1.75)^2\right]$

$= \frac{1}{4}\left[0.0625 + 0.0625 + 3.0625 + 1.5625\right] = \frac{1}{4}[4.75] = 1.1875$

Normalized feature map (Anorm) $= \frac{A - \mu}{\sqrt{\sigma^2}}$

Standard deviation $\sigma = \sqrt{1.1875} = 1.0897$

$$A_{norm} = \frac{A - 1.75}{1.0897}$$

a_{11} and $a_{12} = \frac{2 - 1.75}{1.0897} \approx 0.229$

$a_{21} = \frac{0 - 1.75}{1.0897} \approx -1.605$

$a_{22} = \frac{3 - 1.75}{1.0897} \approx 1.147$

$$A_{norm} = \begin{bmatrix} 0.229 & 0.229 \\ -1.605 & 1.147 \end{bmatrix}$$

Part 3c

3c "Translation invariance" means the network can recognize a feature regardless of exactly where it appears in the image. Our max pooling calculates $Y = \max(A)$. In our forward pass, highest activation was at position a_{22} i.e. 3. This activation of 3 represents the filter detecting specific pattern in bottom-right corner of input image X. If X is shifted one unit to the left, the filter might now detect exact same pattern in the bottom-left corner now instead. So, the value 3 simply moves to a_{21} in A like,

$$A_{shift} = \begin{bmatrix} 2 & 2 \\ 3 & 0 \end{bmatrix}.$$

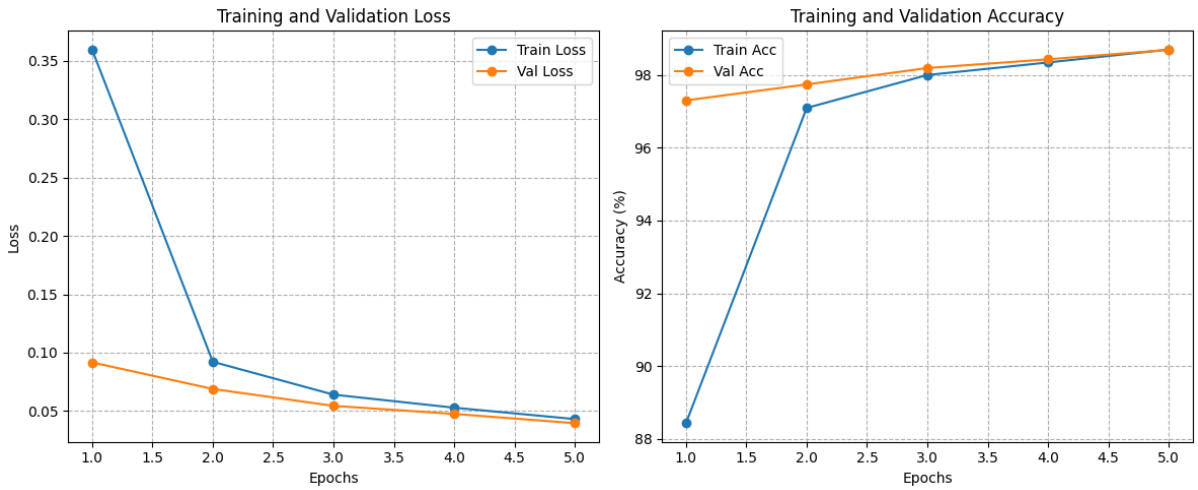
The position may have changed but our max pooling max(A) remains the same which is still 3. So, the output remains the same and so does the loss L.

Programming: Implementation and Analysis

Task 1: Custom CNNs and Shortcut Learning

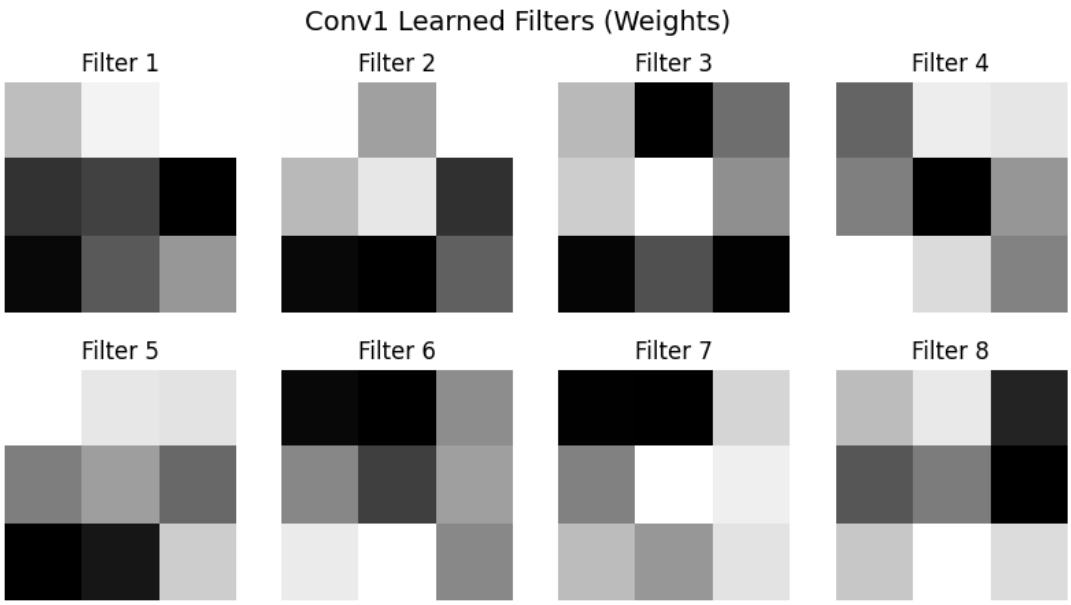
Part A: Standard MNIST

Analytical Question 1.1



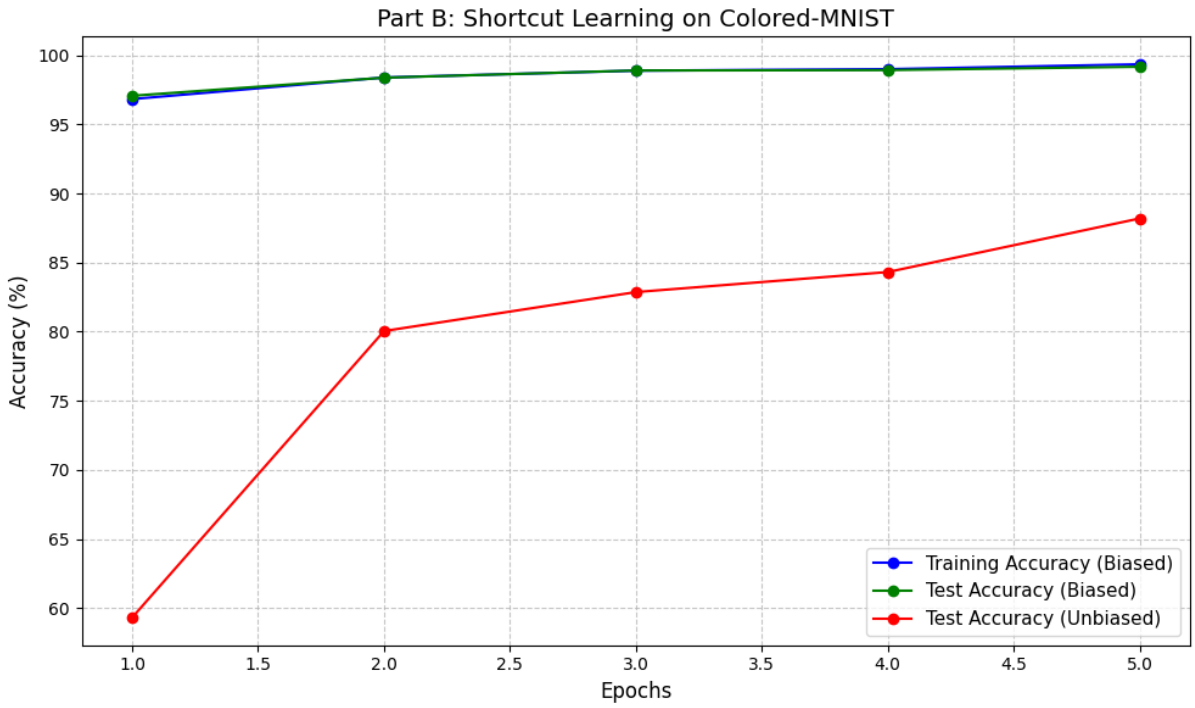
Based on the generated plots, we can see that the model generalized very well. In the loss plot, we can see that both the training and validation loss keep decreasing and converge to a very low value of under 0.05. Moreover, we can see that the validation loss is closely aligned with the training loss which clearly shows that there's no overfitting because if the model was overfitting, the training loss would continue dropping but the validation loss would've spiked. Moreover, in the accuracy plot, we can see that both the training and validation accuracies climb together and both stop together at around 99%. Considering there is no gap between the final training and validation statistics, it shows that our network successfully learned the actual patterns in the data rather than just memorizing the data.

Analytical Question 1.2



Yes, we can visually identify rudimentary edge and gradient detection occurring within these 3x3 weight matrices. In the first layer of the CNN, the network acts as a collection of low-level feature extractors and the early layers capture basic structural changes. Filters 1 and 4 are diagonal detectors as they have contrasting opposite corners, making them sensitive to slanted lines and curves. Filters 5 and 8 are horizontal edge detectors as both of these filters show a strong contrast between horizontal rows, which will heavily activate when sliding over horizontal strokes. Finally, filter 7 is a vertical edge detector because we can see that it shows a clear light-to-dark transition moving from left to right, thus capturing vertical boundaries.

Analytical Question 1.3: Accuracies and Phenomenon Explanation



We can see from the plot that the biased test accuracy is around 99% but the unbiased test accuracy is around 86%.

This significant drop in performance on the unbiased test set happens because the model learned a shortcut. In the biased set, the color is highly correlated with the actual number like zeros are always red and ones are always blue. Neural networks don't run on intuition so they don't know they're supposed to learn shapes and as the color is a reliable predictor, the network just starts learning the color. When we evaluate the model on the unbiased test set, the model's shortcut fails thus leading to a major drop in accuracy.

Mathematically speaking, the weights are updates via gradient descent. The optimizer's goal is to find the fastest path down the loss landscape. Color is a low-variance linearly separable feature and the network only needs to look at a single pixel's RGB channel to detect the color, due to which the gradient pointing towards the color feature is strong and smooth. However, identifying shapes require a complex non-linear aggregation across multiple layers. The optimizer just focuses on the path of least resistance, thus falling into the color shortcut rather than taking the complex mathematical path required to isolate the shape.

Analytical Question 1.4: Proposed Training Strategies

To force the network to ignore the color and focus on the shape instead, several strategies can be used:

1. Data Augmentation: During training, we can apply a function that randomly changes the hue and saturation of every image or we can convert all the images to black and white (grayscale). By randomizing or removing the color, we destroy its predictive value, thus the color no longer minimizes the loss function which forces the optimizer to learn the shape instead.
2. Domain-Adversarial Neural Networks (DANN): We can build a network with two outputs; one that predicts the digit and one that predicts the background color. We can insert a gradient reversal layer between the feature extractor and the color predictor which flips the sign of the gradient during backpropagation. This penalizes the feature extractor if it retains any color information, thus forcing it to learn representations without focusing on color.
3. Reweighting: We can multiple the loss of specific examples where the color doesn't match the standard correlation with a huge weight. This means the network is heavily penalized if it relies on color, thus forcing it to rely on shape instead.

<u>Task 2: Transfer Learning and Interpretability</u>

Part A: Fine-tuning ResNet-18

Analytical Question 2.1

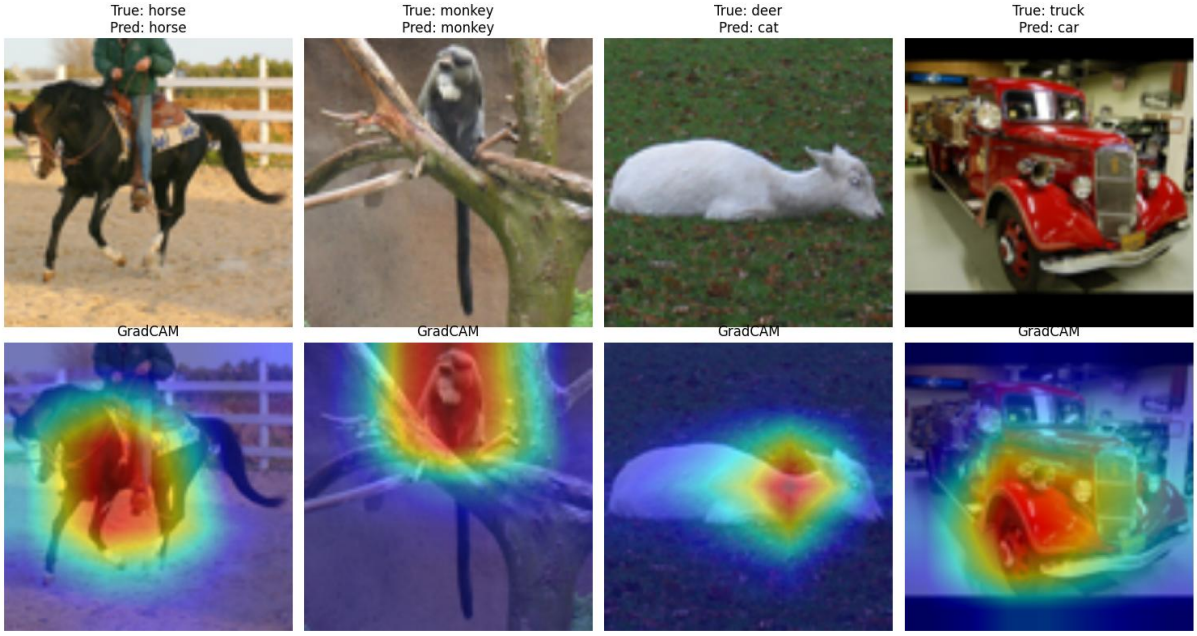
Computationally, it is more beneficial to freeze the early layers because ResNet-18 has over 11 million parameters, so by freezing the "backbone", we tell PyTorch not to calculate gradients or update the weights during backpropagation thus only computing gradients for the single new linear layer at the end. This reduces the memory requires greatly thus making the training exponentially faster.

Functionally speaking, STL-10 is a relatively small dataset compared to ImageNet which has more than 1 million images. If we tried to train all the parameters of ResNet-18 on a small dataset, the model would start to overfit, as it will start to memorize the training images perfectly but fail to generalize. By freezing the network, we restrict the number of trainable parameters thus forcing the model to rely on its pre-existing knowledge of the visual world rather than memorizing the new dataset.

The first few convolutional layers learn very basic, low-level visual elements. In Task 1, we saw that these basic visual elements include edges, lines, color gradients and any simple textures. Of course, the actual physical world is built out of lines and edges so these early features are highly transferable, for example an edge detector learned from a picture of an ImageNet dog can easily be used to detect an edge of an STL-10 picture of an airplane.

However, as we move deeper into the network to the later layers, these simple edges are combined into corners then shapes and eventually complex objects. The last convolutional layers learn highly specific features like wheels of a car or wings of an airplane. These features are highly trained to the original dataset it was trained on and are not transferable at all.

Analytical Question 2.2



As we can see in the first two GradCAM images (the two correct predictions), we can see that the model is focusing on the primary object which integrating some context from the environment. In the first picture, we can see that the model heavily focuses on the horse, that is the primary subject; its main body, legs and neck, so the model is relying on the shape of the animal itself to make the prediction. Similarly, in the second image, the model identifies the monkey's face and upper torso however it is also using some environmental context that is the wooden branch the monkey is sitting on to ensure its prediction is correct.

Analytical Question 2.3

In the last two GradCAM images (the two false predictions), we see that the model fixates on random parts of the image instead of the overall global structure. For example, in the third picture where the deer was misclassified as a cat, the heatmap shows how the network completely ignored the body of the animal and instead just fixated on the ears and the neck. Considering the model isolated these specific features, the head of the deer was easily mistaken as that of a cat. Similarly, in the last image where the truck was misclassified as a car, we can see that the model fixates on the lower front of the vehicle rather than the overall vehicle shape, thus confusing the truck for a simple car.

AI Usage

I made use of Gemini 3 Pro to assist me with PyTorch syntax and some implementation details, particularly while building the custom CNN architectures and also for debugging any errors. Additionally, I used it to clarify some theoretical concepts along the way, specifically backpropagation and shortcut learning. This helped me do a better analysis of my outputs.